

Точни часове и екран „Месец“

В България часовете идват от официалния календар на Главно мюфтийство и съпадат с джамиите до минута. Извън България остава изчислението. Добавени са автоматични тестове и таблица за целия месец.

Календар на Мюфтийството

48 града

Jest – 26 теста

Екран „Месец“

15 файла

Ден	Хиджа	Фажр	Изгрев	Зухр	Аср	Магриб	Иша
1 Чт	20	05:48	07:16	13:22	16:38	19:18	20:39
2 Пт	21	05:49	07:17	13:21	16:37	19:16	20:37
3 Сб	22	05:50	07:18	13:21	16:36	19:14	20:35
4 Нед	23	05:51	07:19	13:21	16:34	19:12	20:34
5 Пн	24	05:53	07:20	13:20	16:33	19:11	20:32
6 Вт	25	05:54	07:21	13:20	16:32	19:09	20:30
7 Ср	26	05:55	07:23	13:20	16:31	19:07	20:28
8 Чт	27	05:56	07:24	13:20	16:29	19:05	20:27
9 Пт	28	05:57	07:25	13:19	16:28	19:04	20:25
10 Сб	29	05:58	07:26	13:19	16:27	19:02	20:23
11 Нед	30	06:00	07:27	13:19	16:25	19:00	20:22
Джумазил-евел 1448							
12 Пн	1	06:01	07:28	13:18	16:24	18:59	20:20
13 Вт	2	06:02	07:29	13:18	16:23	18:57	20:18
14 Ср	3	06:03	07:31	13:18	16:22	18:55	20:17

Какво е решено в този етап

Смесен източник: в България – таблицата на Мюфтийството (София + постоянна разлика за всеки град), извън България – изчисление с adhan (Диянет) · изчислението се разминава с Мюфтийството с 2–4 мин. според годината, затова не се ползва в България · данните са на Мюфтийството – посочени са като източник; преди публикуване – писмо за разрешение.

СЪДЪРЖАНИЕ

01 Защо смесен източник

02 Данните: src/data/mufti.json

03 Логиката: два източника, един вход

04 Автоматични тестове

05 Екранът „Месец“

06 Обновяване на календара

07 Проверка и край на етапа

Стъпка 1

Защо смесен източник

Първоначалният план беше изчисление с `adhan` плюс постоянни корекции. Сравнението с пълната таблица на Мюфтийството (365 дни) показва, че това не стига за точност ± 1 минута.

МОЛИТВА	НАЙ-ГОЛЯМА РАЗЛИКА (2026)	ДНИ ДО 1 МИН.
Фаджр	2 мин.	93%
Изгрев	2 мин.	97%
Зухр	1 мин.	100%
Аср	3 мин.	64%
Магриб	3 мин.	59%
Иша	3 мин.	38%

- Методът „Turkey“ в `adhan` вече съдържа същите предпазни минути (темкин) като Мюфтийството – средната разлика е ~ 0 .
- Таблицата на Мюфтийството е „вечен“ календар – една и съща всяка година. Слънцето спрямо григорианския календар се мести с до четвърт ден в 4-годишния цикъл на високосната година, затова разликата е 2 мин. във високосна година и до 4 мин. в следващата. Не расте с десетилетията.
- Всички 48 града в календара са **таблицата на София + постоянна разлика в минути** (Варна –18, Кърджали –8...), приблизително 4 минути на градус географска дължина.

Решение: в България – официалната таблица (15 KB, работи офлайн, съвпада до минута); извън България – изчисление с `adhan`.

Стъпка 2

Данните: `src/data/mufti.json`

Един файл с таблицата на София и 48-те града. Не се пише на ръка – генерира се от сайта на Мюфтийството (стъпка 7).

Структура (съкратено; в JSON няма коментари)

```
{
  "source": "Главно мюфтийство на Република България - https://www.grandmufti.bg/...",
  "retrieved": "2026-09-25",
  "columns": ["fajr", "sunrise", "dhuhr", "asr", "maghrib", "isha"],
  "sofia": [
    [374, 470, 755, 889, 1030, 1119], // 1 януари: 06:14 07:50 12:35 14:49 17:10 18:39
    [374, 470, 755, 890, 1030, 1120], // 2 януари
    ...                               // 365 реда
  ],
  "towns": [
    { "id": "varna", "bg": "Варна", "en": "Varna", "lat": 43.2074, "lon": 27.9167, "shift": -18 },
    ...                               // 48 града
  ]
}
```

- Часовете са **минути от полунощ в зимно време (UTC+2)**. Лятното време не е в таблицата – добавя го телефонът според часовата зона. Така един и същ файл важи за всяка година, независимо кога точно се сменя часът.
- 29 февруари липсва (365 реда). За него се взима средното от 28 февруари и 1 март.
- Английските имена на градовете са по официалната транслитерация (`Kardzhali` , `Svishtov`), с изключение на `Sofia` .

Стъпка 3

Логиката: два източника, един вход

`prayers.ts` вече не изчислява нищо – само знае коя молитва е сега и коя е следващата. Часовете идват от три нови файла.

Дърво на проекта

```
src/domain/
├─ location.ts  AppLocation + source: 'mufti' | 'calc'
├─ mufti.ts     календарът на Мюфтийството, градове, най-близък град
├─ calc.ts      изчисление с adhan (извън България)
├─ times.ts     computeDay – единственото място, което избира източника
└─ prayers.ts   молитвите и „сега / следваща“ (без промяна в логиката)
```

`location.ts` src/domain

- `source` решава откъде идват часовете. `muftiShift` е разликата спрямо София (само за България).
- Градовете в етап 3 (избор и GPS) ще създават точно такъв обект.

src/domain/location.ts

17 реда

```
/**
 * Откъде идват часовете за едно място:
 * - 'mufti' - официалният календар на Главно мюфтийство (всички места в България);
 * - 'calc' - астрономическо изчисление с adhan (извън България).
 */
export type TimeSource = 'mufti' | 'calc';

export interface AppLocation {
  id: string;
  /** Името на мястото на двата езика (градовете са данни, а не текстове на интерфейса). */
  names: { bg: string; en: string };
  latitude: number;
  longitude: number;
  source: TimeSource;
  /** Само за 'mufti': разлика в минути спрямо таблицата на София (Варна = -18). */
  muftiShift?: number;
}
```

`mufti.ts` src/domain

- `computeMuftiDay`: минутите са в UTC+2, затова `Date.UTC(..., минути + shift - 120)` дава точния момент. Телефонът го показва в своята часова зона – с лятно време, когато го има. Няма нито един ред код за лятно време.

- `muftiShiftFor` е за места извън списъка (село, GPS): най-близкият град + 4 минути на градус дължина. Същото правило ползва и Мюфтийството.

src/domain/mufti.ts

104 реда

```
import data from '@data/mufti.json';

import type { AppLocation } from './location';
import { PRAYER_IDS, type PrayerTime } from './prayers';

/**
 * Официалният календар на Главно мюфтийство.
 *
 * Как е устроен (виж src/data/mufti.json):
 * - `sofia` е таблица 365 × 6 – минути от полунощ за Фаджр, Изгрев, Зухр, Аср, Магриб, Иша,
 *   винаги в ЗИМНО време (UTC+2). Лятното време добавя телефонът според часовата зона.
 * - Всеки от 48-те града = таблицата на София + постоянна разлика `shift` в минути.
 * - Календарът е един и същ всяка година („вечен“ календар).
 */

export interface MuftiTown {
  id: string;
  bg: string;
  en: string;
  lat: number;
  lon: number;
  shift: number;
}

export const MUFTI_TOWNS: readonly MuftiTown[] = data.towns;
export const MUFTI_SOURCE: string = data.source;
const SOFIA: readonly (readonly number[])[] = data.sofia;

const DAYS_BEFORE_MONTH = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334];

/** Редът от таблицата за дадена дата. 29 февруари го няма – взимаме средното на 28.02 и 01.03. */
function rowFor(date: Date): readonly number[] {
  const month = date.getMonth();
  const day = date.getDate();
  if (month === 1 && day === 29) {
    const feb28 = SOFIA[58];
    const mar1 = SOFIA[59];
    return feb28.map((v, k) => Math.round((v + mar1[k]) / 2));
  }
  return SOFIA[DAYS_BEFORE_MONTH[month] + day - 1];
}

/**
 * Часовете за един ден по календара на Мюфтийството.
 * Минутите са в UTC+2, затова UTC = минути - 120. Така се получава точен момент във времето,
 * който телефонът показва в своята часова зона (с лятното време, когато го има).
 */
export function computeMuftiDay(date: Date, shift: number): PrayerTime[] {
  const row = rowFor(date);
  const y = date.getFullYear();
  const m = date.getMonth();
  const d = date.getDate();
  return PRAYER_IDS.map((id, k) => ({
    id,
    time: new Date(Date.UTC(y, m, d, 0, row[k] + shift - 120)),
  }));
}
```

```

}

export function townToLocation(town: MuftiTown): AppLocation {
  return {
    id: town.id,
    names: { bg: town.bg, en: town.en },
    latitude: town.lat,
    longitude: town.lon,
    source: 'mufti',
    muftiShift: town.shift,
  };
}

export function findTown(id: string): MuftiTown | undefined {
  return MUFTI_TOWNS.find((t) => t.id === id);
}

function distanceKm(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const rad = Math.PI / 180;
  const dLat = (lat2 - lat1) * rad;
  const dLon = (lon2 - lon1) * rad;
  const a =
    Math.sin(dLat / 2) ** 2 + Math.cos(lat1 * rad) * Math.cos(lat2 * rad) * Math.sin(dLon / 2) ** 2;
  return 6371 * 2 * Math.asin(Math.sqrt(a));
}

export function nearestTown(lat: number, lon: number): { town: MuftiTown; km: number } {
  let best = MUFTI_TOWNS[0];
  let bestKm = Infinity;
  for (const t of MUFTI_TOWNS) {
    const km = distanceKm(lat, lon, t.lat, t.lon);
    if (km < bestKm) {
      best = t;
      bestKm = km;
    }
  }
  return { town: best, km: bestKm };
}

/**
 * Разликата за място в България, което не е в списъка (село, GPS):
 * най-близкият град + 4 минути за всеки градус географска дължина (на изток - по-рано).
 * Същото правило ползва и самото Мюфтийство за 48-те града.
 */
export function muftiShiftFor(lat: number, lon: number): number {
  const { town } = nearestTown(lat, lon);
  return town.shift - Math.round((lon - town.lon) * 4);
}

```

calc.ts src/domain

- Кодът за `adhan` от етап 0, преместен в отделен файл. Ползва се само за `source: 'calc'`.

src/domain/calc.ts

28 реда

```

import { CalculationMethod, Coordinates, Madhab, PrayerTimes } from 'adhan';

import type { AppLocation } from '../location';
import { PRAYER_IDS, type PrayerTime } from '../prayers';

```

```

export type CalculationMethodId = 'Turkey' | 'MuslimWorldLeague' | 'Egyptian' | 'UmmAlQura';
export type AsrMadhab = 'shafi' | 'hanafi';

/**
 * Астрономическо изчисление с adhan – за места извън България.
 * Методът „Turkey“ (Диянет) вече съдържа предпазните минути (темкин) на Диянет.
 * Сравнено с календара на Мюфтийството за София, средната разлика е ~0,
 * но в отделни дни стига 2-4 мин. (виж docs/PLAN.md, етап 2).
 */
export function computeCalcDay(
  date: Date,
  location: AppLocation,
  method: CalculationMethodId,
  madhab: AsrMadhab,
): PrayerTime[] {
  const params = CalculationMethod[method]();
  params.madhab = madhab === 'hanafi' ? Madhab.Hanafi : Madhab.Shafi;
  const coords = new Coordinates(location.latitude, location.longitude);
  // обяд, за да не може смяната на лятно/зимно време да измести датата
  const noon = new Date(date.getFullYear(), date.getMonth(), date.getDate(), 12);
  const pt = new PrayerTimes(coords, noon, params);
  return PRAYER_IDS.map((id) => ({ id, time: pt[id] }));
}

```

times.ts src/domain

- `computeDay` избира източника и прилага ръчните корекции (`offsets`) – те ще се настройват в етап 3.
- `computeMonth` връща всички дни от месеца за екрана „Месец“.

src/domain/times.ts

58 реда

```

import { computeCalcDay, type AsrMadhab, type CalculationMethodId } from './calc';
import type { AppLocation } from './location';
import { computeMuftiDay } from './mufti';
import type { PrayerId, PrayerTime, ThreeDays } from './prayers';

export interface TimesOptions {
  location: AppLocation;
  /** Само за source 'calc'. */
  method: CalculationMethodId;
  /** Само за source 'calc'. В България Аср е винаги по календара на Мюфтийството. */
  madhab: AsrMadhab;
  /** Ръчни корекции в минути (+/-) за всяка молитва. Настройва се в етап 3. */
  offsets?: Partial<Record<PrayerId, number>>;
}

const MINUTE = 60_000;

/** Единственото място, което решава откъде идват часовете за даден ден. */
export function computeDay(date: Date, opts: TimesOptions): PrayerTime[] {
  const { location } = opts;
  const base =
    location.source === 'mufti'
      ? computeMuftiDay(date, location.muftiShift ?? 0)
      : computeCalcDay(date, location, opts.method, opts.madhab);

  if (!opts.offsets) return base;
  return base.map((p) => {
    const off = opts.offsets?.[p.id] ?? 0;
    return off ? { id: p.id, time: new Date(p.time.getTime() + off * MINUTE) } : p;
  });
}

```

```

});
}

const dayAt = (d: Date, offset: number) =>
  new Date(d.getFullYear(), d.getMonth(), d.getDate() + offset, 12);

export function computeThreeDays(now: Date, opts: TimesOptions): ThreeDays {
  return {
    yesterday: computeDay(dayAt(now, -1), opts),
    today: computeDay(dayAt(now, 0), opts),
    tomorrow: computeDay(dayAt(now, 1), opts),
  };
}

export interface MonthDay {
  date: Date; // обяд на съответния ден
  times: PrayerTime[];
}

/** Всички дни от месеца (month: 0 = януари). */
export function computeMonth(year: number, month: number, opts: TimesOptions): MonthDay[] {
  const days = new Date(year, month + 1, 0).getDate();
  const out: MonthDay[] = [];
  for (let d = 1; d <= days; d++) {
    const date = new Date(year, month, d, 12);
    out.push({ date, times: computeDay(date, opts) });
  }
  return out;
}

```

defaults.ts src/config

- Подразбиращото се място вече е София от календара на Мюфтийството (`source: 'mufti'`).

src/config/defaults.ts

17 реда

```

import type { AsrMadhab, CalculationMethodId } from '@domain/calc';
import type { AppLocation } from '@domain/location';
import { findTown, townToLocation } from '@domain/mufti';

/**
 * Стойности по подразбиране, докато няма екран за настройки и GPS (етап 3 от плана).
 */

/** София от календара на Мюфтийството. */
export const DEFAULT_LOCATION: AppLocation = townToLocation(findTown('sofia')!);

/**
 * За места извън България (source 'calc'): метод Диянет и „първият“ Аср (Шафии),
 * както в календарите на Диянет и на Мюфтийството. Ханафи дава Аср около час по-късно.
 */
export const DEFAULT_METHOD: CalculationMethodId = 'Turkey';
export const DEFAULT_MADHAB: AsrMadhab = 'shafi';

```

usePrayerSchedule.ts src/hooks

- `TIMES_OPTIONS` се експортира, защото и екранът „Месец“ го ползва. В етап 3 ще идва от настройките.

src/hooks/usePrayerSchedule.ts

25 реда

```
import { useMemo } from 'react';

import { DEFAULT_LOCATION, DEFAULT_MADHAB, DEFAULT_METHOD } from '@config/defaults';
import { getSchedule, type Schedule } from '@domain/prayers';
import { computeThreeDays, type TimesOptions } from '@domain/times';

/** Засега фиксирани; в етап 3 идват от настройките (град, метод, корекции). */
export const TIMES_OPTIONS: TimesOptions = {
  location: DEFAULT_LOCATION,
  method: DEFAULT_METHOD,
  madhab: DEFAULT_MADHAB,
};

/**
 * Часовете се преизчисляват само когато се смени денят,
 * а текущата/следващата молитва – на всяка секунда (евтино е).
 */
export function usePrayerSchedule(now: Date): Schedule {
  const dayKey = `${now.getFullYear()}-${now.getMonth()}-${now.getDate()}`;

  // eslint-disable-next-line react-hooks/exhaustive-deps
  const days = useMemo(() => computeThreeDays(now, TIMES_OPTIONS), [dayKey]);

  return getSchedule(now, days);
}
```

Стъпка 4

Автоматични тестове

Логиката в `src/domain` е чист TypeScript, затова се тества без телефон.

Jest с настройките на Expo

```
npx expo install -- --save-dev jest-expo jest
npm install --save-dev "@react-native/jest-preset@~0.86.3" "@types/jest@^29.5.14"
```

package.json (частта за Jest)

```
"scripts": { "test": "jest", ... },
"jest": {
  "preset": "jest-expo",
  "testMatch": ["**/__tests__/**/*.test.ts"],
  "globalSetup": "./jest.global-setup.js"
}
```

В `tsconfig.json` е добавено `"types": ["jest"]`. TypeScript 6 вече не зарежда глобалните типове автоматично и без това `describe` / `expect` са непознати.

jest.global-setup.js корен на проекта

- Тестовите проверяват часове в София, затова часовата зона е фиксирана – резултатът е един и същ на всеки компютър.

jest.global-setup.js

5 реда

```
// Тестовите проверяват часове в София, затова часовата зона е фиксирана
// независимо от компютъра, на който се пускат.
module.exports = () => {
  process.env.TZ = 'Europe/Sofia';
};
```

mufti.test.ts src/domain/__tests__

- Стойностите за 1 и 29 септември са преписани от сайта на Мюфтийството – независима проверка, а не сравнение на файла със самия себе си.
- Проверява и смяната на лятно време, 29 февруари, реда на часовете за всички 366 дни на високосна година, Варна и най-близкия град.

src/domain/__tests__/mufti.test.ts

91 реда

```
import { formatHM } from '../format';
import { computeMuftiDay, findTown, muftiShiftFor, nearestTown, townToLocation } from '../mufti';
import { computeDay } from '../times';

/** Часовете за деня като ['05:45', '07:14', ...] в часовата зона на теста (Europe/Sofia). */
const hm = (y: number, m: number, d: number, shift = 0) =>
  computeMuftiDay(new Date(y, m - 1, d, 12), shift).map((p) => formatHM(p.time));

describe('календар на Мюфтийството – София', () => {
  // Стойностите са преписани от grandmufti.bg (таблица „Времена за намаз“, София, септември).
  it('29 септември съвпада със сайта (лятно време)', () => {
    expect(hm(2026, 9, 29)).toEqual(['05:45', '07:14', '13:22', '16:41', '19:21', '20:43']);
  });

  it('1 септември съвпада със сайта', () => {
    expect(hm(2026, 9, 1)).toEqual(['05:09', '06:43', '13:32', '17:13', '20:11', '21:38']);
  });

  it('същата дата дава същите часове в друга година (вечен календар)', () => {
    expect(hm(2031, 9, 29)).toEqual(hm(2026, 9, 29));
  });

  it('смяната на лятно време мести часовете с един час', () => {
    const before = hm(2026, 3, 28); // последният ден зимно време
    const after = hm(2026, 3, 29); // първият ден лятно време
    const minutes = (s: string) => Number(s.slice(0, 2)) * 60 + Number(s.slice(3));
    const jump = minutes(after[2]) - minutes(before[2]); // Зухр се мени плавно
    expect(jump).toBeGreaterThanOrEqual(59);
    expect(jump).toBeLessThanOrEqual(60);
  });

  it('29 февруари е между 28 февруари и 1 март', () => {
    const mins = (d: number, m: number) =>
      computeMuftiDay(new Date(2028, m - 1, d, 12), 0).map(
        (p) => p.time.getHours() * 60 + p.time.getMinutes(),
      );
    const feb28 = mins(28, 2);
```

```

const feb29 = mins(29, 2);
const mar1 = mins(1, 3);
feb29.forEach((v, k) => {
  expect(v).toBeGreaterThanOrEqual(Math.min(feb28[k], mar1[k]));
  expect(v).toBeLessThanOrEqual(Math.max(feb28[k], mar1[k]));
});
});

it('часовете вървят в правилния ред всеки ден от годината', () => {
  for (let i = 0; i < 366; i++) {
    const day = computeMuftiDay(new Date(2028, 0, 1 + i, 12), 0).map((p) => p.time.getTime());
    for (let k = 1; k < day.length; k++) expect(day[k]).toBeGreaterThan(day[k - 1]);
  }
});

describe('други градове', () => {
  it('Варна е София -18 минути', () => {
    expect(findTown('varna')?.shift).toBe(-18);
    expect(hm(2026, 9, 29, -18)).toEqual(['05:27', '06:56', '13:04', '16:23', '19:03', '20:25']);
  });

  it('computeDay използва календара за град от списъка', () => {
    const varna = townToLocation(findTown('varna')!);
    const times = computeDay(new Date(2026, 8, 29, 12), {
      location: varna,
      method: 'Turkey',
      madhab: 'shafi',
    }).map((p) => formatHM(p.time));
    expect(times[0]).toBe('05:27');
  });

  it('всички 48 града следват правилото „4 минути на градус“ (±2 мин.)', () => {
    const sofia = findTown('sofia')!;
    for (const id of ['varna', 'burgas', 'kardjali', 'ruse', 'shumen', 'plovdiv', 'blagoevgrad']) {
      const t = findTown(id)!;
      const estimate = -(t.lon - sofia.lon) * 4;
      expect(Math.abs(estimate - t.shift)).toBeLessThanOrEqual(2);
    }
  });
});

describe('място извън списъка (село, GPS)', () => {
  it('в самия град дава неговата разлика', () => {
    const v = findTown('varna')!;
    expect(muftiShiftFor(v.lat, v.lon)).toBe(-18);
  });

  it('Момчилград е най-близо до Кърджали', () => {
    const { town } = nearestTown(41.527, 25.408);
    expect(town.id).toBe('kardjali');
    expect(muftiShiftFor(41.527, 25.408)).toBe(-8);
  });
});

```

schedule.test.ts src/domain/__tests__

- Граничните случаи: преди Фаджр, след Иша, точно в часа на молитвата, двете нощи със смяна на часа.

src/domain/__tests__/schedule.test.ts

68 реда

```
import { DEFAULT_LOCATION } from '@config/defaults';

import { getSchedule } from '../prayers';
import { computeThreeDays, type TimesOptions } from '../times';

const opts: TimesOptions = { location: DEFAULT_LOCATION, method: 'Turkey', madhab: 'shafi' };
const at = (y: number, m: number, d: number, h: number, min: number) => new Date(y, m - 1, d, h, min);
const schedule = (now: Date) => getSchedule(now, computeThreeDays(now, opts));

describe('текуща и следваща молитва (София, 29.09.2026)', () => {
  // Фаджр 05:45, Изгрев 07:14, Зухр 13:22, Аср 16:41, Магриб 19:21, Иша 20:43

  it('следобед: сега е Аср, следва Магриб', () => {
    const s = schedule(at(2026, 9, 29, 17, 0));
    expect(s.current.id).toBe('asr');
    expect(s.next.id).toBe('maghrib');
    expect(s.isNextTomorrow).toBe(false);
    expect(s.remainingMs).toBe((2 * 60 + 21) * 60_000);
  });

  it('преди Фаджр: сега е вчерашната Иша, следва днешният Фаджр', () => {
    const s = schedule(at(2026, 9, 29, 3, 0));
    expect(s.current.id).toBe('isha');
    expect(s.current.time.getDate()).toBe(28);
    expect(s.next.id).toBe('fajr');
    expect(s.isNextTomorrow).toBe(false);
    expect(s.rowState('isha')).toBe('now');
    expect(s.rowState('fajr')).toBe('upcoming');
  });

  it('след Иша: следва утрешният Фаджр', () => {
    const s = schedule(at(2026, 9, 29, 22, 30));
    expect(s.current.id).toBe('isha');
    expect(s.next.id).toBe('fajr');
    expect(s.next.time.getDate()).toBe(30);
    expect(s.isNextTomorrow).toBe(true);
  });

  it('точно в часа на молитвата тя вече е текуща', () => {
    const s = schedule(at(2026, 9, 29, 13, 22));
    expect(s.current.id).toBe('dhuhr');
    expect(s.rowState('sunrise')).toBe('past');
  });

  it('прогресът е между 0 и 1', () => {
    for (let h = 0; h < 24; h++) {
      const p = schedule(at(2026, 9, 29, h, 17)).progress;
      expect(p).toBeGreaterThanOrEqual(0);
      expect(p).toBeLessThanOrEqual(1);
    }
  });

  describe('смяна на часовото време', () => {
    it('нощта към лятно време (29.03.2026) не обърква следващата молитва', () => {
```

```

const s = schedule(at(2026, 3, 29, 2, 30)); // 02:30 - преди прескачането 03:00 → 04:00
expect(s.current.id).toBe('isha');
expect(s.next.id).toBe('fajr');
expect(s.next.time.getDate()).toBe(29);
});

it('нощта към зимно време (25.10.2026) не обърква следващата молитва', () => {
  const s = schedule(at(2026, 10, 25, 3, 30));
  expect(s.next.id).toBe('fajr');
  expect(s.next.time.getDate()).toBe(25);
  expect(s.remainingMs).toBeGreaterThan(0);
});
});

```

calc.test.ts src/domain/__tests__

- Документира числото, заради което ползваме таблицата: изчислението се разминава с Мюфтийството с до 4 мин.

src/domain/__tests__/calc.test.ts

40 реда

```

import { computeCalcDay } from '../calc';
import type { AppLocation } from '../location';
import { computeMuftiDay } from '../mufti';

const sofiaCalc: AppLocation = {
  id: 'sofia-calc',
  names: { bg: 'София', en: 'Sofia' },
  latitude: 42.6977,
  longitude: 23.3217,
  source: 'calc',
};

describe('изчисление с adhan (извън България)', () => {
  it('часовете вървят в правилния ред', () => {
    const istanbul: AppLocation = { ...sofiaCalc, id: 'ist', latitude: 41.0082, longitude: 28.9784 };
    const t = computeCalcDay(new Date(2026, 5, 21, 12), istanbul, 'Turkey', 'shafi').map((p) =>
      p.time.getTime());
    for (let k = 1; k < t.length; k++) expect(t[k]).toBeGreaterThan(t[k - 1]);
  });

  it('Ханафи дава по-късен Аср от Шафии', () => {
    const date = new Date(2026, 5, 21, 12);
    const shafi = computeCalcDay(date, sofiaCalc, 'Turkey', 'shafi')[3].time.getTime();
    const hanafi = computeCalcDay(date, sofiaCalc, 'Turkey', 'hanafi')[3].time.getTime();
    expect(hanafi - shafi).toBeGreaterThan(30 * 60_000);
  });

  // Документира защо в България ползваме таблицата: изчислението се разминава с до ~4 мин.
  it('за София се разминава с Мюфтийството с най-много 4 минути през 2026', () => {
    let worst = 0;
    for (let i = 0; i < 365; i++) {
      const date = new Date(2026, 0, 1 + i, 12);
      const calc = computeCalcDay(date, sofiaCalc, 'Turkey', 'shafi');
      const mufti = computeMuftiDay(date, 0);
      calc.forEach((p, k) => {
        worst = Math.max(worst, Math.abs(p.time.getTime() - mufti[k].time.getTime()) / 60_000);
      });
    }
    expect(worst).toBeLessThanOrEqual(4.5);
  });
}

```

```
});  
});
```

dates.test.ts src/domain/__tests__

- Форматиране на двата езика и датата по Хиджра (Рамазан байрам 2026 = 1 Шеввал 1447).

src/domain/__tests__/dates.test.ts

43 реда

```
import { bg, en } from '@i18n/strings';  
  
import { formatCountdown, formatGregorianShort, formatHM } from '../format';  
import { formatHijri, tabularHijri, toHijri } from '../hijri';  
  
describe('форматиране', () => {  
  it('час и обратно броене', () => {  
    expect(formatHM(new Date(2026, 8, 29, 7, 5))).toBe('07:05');  
    expect(formatCountdown(((1 * 60 + 49) * 60 + 12) * 1000)).toBe('01:49:12');  
    expect(formatCountdown(500)).toBe('00:00:01'); // закръгля нагоре  
    expect(formatCountdown(-1000)).toBe('00:00:00');  
  });  
  
  it('дата на двата езика', () => {  
    const d = new Date(2026, 8, 29, 12);  
    expect(formatGregorianShort(d, bg.date)).toBe('Вт, 29 септ.');
```

expect(formatGregorianShort(d, en.date)).toBe('Tue, 29 Sep');

expect(formatGregorianShort(new Date(2026, 2, 20, 12), bg.date)).toBe('Пт, 20 март');

});

});

```
describe('дата по Хиджра', () => {  
  it('Рамазан байрам 2026 е 1 Шеввал 1447', () => {  
    const h = toHijri(new Date(2026, 2, 20, 12));  
    expect([h.day, h.month, h.year]).toEqual([1, 10, 1447]);  
  });  
  
  it('табличният алгоритъм е в рамките на ±2 дни от Umm al-Qura', () => {  
    for (let i = 0; i < 400; i += 7) {  
      const d = new Date(2026, 0, 1 + i, 12);  
      const a = toHijri(d);  
      const b = tabularHijri(d);  
      const sameMonth = a.year === b.year && a.month === b.month;  
      if (sameMonth) expect(Math.abs(a.day - b.day)).toBeLessThanOrEqual(2);  
    }  
  });  
  
  it('корекцията мести датата с ден', () => {  
    const d = new Date(2026, 8, 29, 12);  
    expect(toHijri(d, 1).day).toBe((toHijri(d).day % 30) + 1);  
    expect(formatHijri(d, en.hijriMonths)).toMatch(/^d{1,2} [A-Za-z' -]+ 1448$/);  
  });  
});
```

Трябва да видиш: Tests: 26 passed, 26 total

npm test

Стъпка 5

Екранът „Месец“

Таблица с шестте часа за всеки ден от месеца, по същия дизайн като „Днес“.

- Стрелките сменят месеца. Под името на месеца – месеците по Хиджра, които обхваща.
- Днешният ден е в злато и таблицата се превърта до него. В друг месец се появява бутон „Днес“.
- Под всяка дата – денят по Хиджра (в заглавния ред под „Ден“ пише „Хиджра“). Там, където започва нов месец по Хиджра, има разделителен ред с името му, напр. „Джемазиел-евел 1448“ (избран вариант D от moskur-a). Петък е отбелязан със златен ден от седмицата.
- Най-долу – източникът: „Главно мюфтийство на Република България“ или „Изчислено по метод Диянет“.

src/i18n/strings.ts – нови текстове (същите ключове и в en)

```
month: {
  dayColumn: 'Ден',
  hijriColumn: 'Хиджра',
  today: 'Днес',
  prev: 'Предишен месец',
  next: 'Следващ месец',
  friday: 'петък',
  sourceMufti: 'Източник: Главно мюфтийство на Република България',
  sourceCalc: 'Изчислено по метод Диянет',
},
// в date: пълните имена на месеците
monthsFull: ['Януари', 'Февруари', 'Март', ...],
```

src/components/icons.tsx – нова икона за стрелките

```
export function ChevronIcon({ size = 20, color, direction }: IconProps & { direction: 'left' | 'right' }) {
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Path d={direction === 'left' ? 'M15 6l-6 6 6 6' : 'M9 6l6 6 6 6'} {...stroke(color, 2.2)} />
    </Svg>
  );
}
```

MonthHeader.tsx src/components

- Заглавие, стрелки, диапазон по Хиджра, град и бутон „Днес“.

src/components/MonthHeader.tsx

96 реда

```
import { Pressable, StyleSheet, Text, View } from 'react-native';

import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { ChevronIcon } from '../icons';

interface Props {
  title: string; // „Октомври 2026“
  hijriRange: string; // „Ребиул-ахир – Джемазиел-евел 1448“
  city: string;
  showToday: boolean;
  onPrev: () => void;
```

```

onNext: () => void;
onToday: () => void;
}

/** Месец и година, стрелки за смяна, диапазон по Хиджра и бутон „Днес“. */
export function MonthHeader({ title, hijriRange, city, showToday, onPrev, onNext, onToday }: Props) {
  const { t } = useI18n();

  return (
    <View style={styles.wrap}>
      <View style={styles.row}>
        <Pressable
          onPress={onPrev}
          hitSlop={8}
          style={({ pressed }) => [styles.arrow, pressed && styles.pressed]}
          accessibilityRole="button"
          accessibilityLabel={t.month.prev}
        >
          <ChevronIcon direction="left" color={colors.text} />
        </Pressable>
        <View style={styles.titles}>
          <Text style={styles.title}>{title}</Text>
          <Text style={styles.hijri}>{hijriRange}</Text>
        </View>
        <Pressable
          onPress={onNext}
          hitSlop={8}
          style={({ pressed }) => [styles.arrow, pressed && styles.pressed]}
          accessibilityRole="button"
          accessibilityLabel={t.month.next}
        >
          <ChevronIcon direction="right" color={colors.text} />
        </Pressable>
      </View>
      <View style={styles.meta}>
        <Text style={styles.city}>{city}</Text>
        {showToday && (
          <Pressable
            onPress={onToday}
            style={({ pressed }) => [styles.todayBtn, pressed && styles.pressed]}
            accessibilityRole="button"
          >
            <Text style={styles.todayText}>{t.month.today}</Text>
          </Pressable>
        )}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: { paddingHorizontal: 14, paddingTop: 10, paddingBottom: 12, gap: 8 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  arrow: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: colors.pill,
  },

```

```

pressed: { opacity: 0.7 },
titles: { flex: 1, alignItems: 'center' },
title: { fontFamily: fonts.extrabold, fontSize: 22, color: colors.text },
hijri: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.gold, textAlign: 'center' },
meta: {
  flexDirection: 'row',
  justifyContent: 'center',
  alignItems: 'center',
  gap: 10,
  minHeight: 28,
},
city: { fontFamily: fonts.semibold, fontSize: 13, color: colors.muted },
todayBtn: {
  paddingHorizontal: 12,
  paddingVertical: 4,
  borderRadius: 999,
  borderWidth: 1,
  borderColor: colors.gold,
},
todayText: { fontFamily: fonts.bold, fontSize: 12, color: colors.gold },
});

```

MonthTable.tsx src/components

- Заглавният ред, разделителят за нов месец по Хиджра (`HijriMonthSeparator`) и един ред за ден.
- Фиксираните височини на реда и на разделителя (`MONTH_ROW_HEIGHT` , `HIJRI_SEPARATOR_HEIGHT`) позволяват да се превърти точно до днешния ден.
- `adjustsFontSizeToFit` в заглавния ред смалява „Магриб“/„Sunrise“, ако на малък телефон не се събират.

src/components/MonthTable.tsx

118 реда

```

import { StyleSheet, Text, View } from 'react-native';

import { formatHM } from '@/domain/format';
import { PRAYER_IDS } from '@/domain/prayers';
import type { MonthDay } from '@/domain/times';
import { useI18n } from '@/i18n';
import { colors } from '@/theme/colors';
import { fonts, tabularNums } from '@/theme/typography';

export const MONTH_ROW_HEIGHT = 42;
/** Височината на разделителя за нов месец по Хиджра (26 + 2 × 2 отстъп). */
export const HIJRI_SEPARATOR_HEIGHT = 30;
const DAY_COL = 50;

/** Заглавният ред: „Ден / Хиджра“ и имената на молитвите. */
export function MonthColumns() {
  const { t } = useI18n();
  return (
    <View style={[styles.row, styles.headRow]}>
      <View style={styles.dayCol}>
        <Text style={[styles.head, styles.headLeft]}>{t.month.dayColumn}</Text>
        <Text style={styles.headHijri}>{t.month.hijriColumn}</Text>
      </View>
      {PRAYER_IDS.map((id) => (
        <Text key={id} style={[styles.cell, styles.head]} numberOfLines={1} adjustsFontSizeToFit>
          {t.prayers[id]}
        </Text>
      ))}
    </View>
  );
}

```

```

    )})
  </View>
);
}

/** Разделител на цялата ширина там, където започва нов месец по Хиджра: — Джемазиел-евел 1448 — */
export function HijriMonthSeparator({ label }: { label: string }) {
  return (
    <View style={styles.separator} accessibilityRole="header">
      <View style={styles.sepLine} />
      <Text style={styles.sepText}>{label}</Text>
      <View style={styles.sepLine} />
    </View>
  );
}

interface RowProps {
  day: MonthDay;
  hijriDay: number;
  isToday: boolean;
}

/** Един ден: дата + ден от седмицата, ден по Хиджра и шестте часа. Петък е със златен ден от седмицата. */
export function MonthRow({ day, hijriDay, isToday }: RowProps) {
  const { t } = useI18n();
  const d = day.date;
  const isFriday = d.getDay() === 5;

  return (
    <View
      style={[styles.row, styles.dayRow, isToday && styles.today]}
      accessibilityLabel={isFriday ? `${d.getDate()}, ${t.month.friday}` : undefined}
    >
      <View style={styles.dayCol}>
        <View style={styles.dateLine}>
          <Text style={[styles.dateNum, isToday && styles.gold]}>{d.getDate()}</Text>
          <Text style={[styles.weekday, isFriday && styles.gold]}>{t.date.weekdaysShort[d.getDay()]}</Text>
        </View>
        <Text style={styles.hijriDay}>{hijriDay}</Text>
      </View>
      {day.times.map((p) => (
        <Text key={p.id} style={[styles.cell, styles.time, isToday && styles.timeToday]}>
          {formatHM(p.time)}
        </Text>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', paddingHorizontal: 8 },
  headRow: {
    height: 36,
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: 'rgba(255,255,255,0.14)',
  },
  head: {
    fontFamily: fonts.bold,
    fontSize: 10.5,
    letterSpacing: 0.3,
    color: colors.muted,
  }
});

```

```

    textAlign: 'center',
  },
  headLeft: { textAlign: 'left' },
  headHijri: { fontFamily: fonts.semibold, fontSize: 9, color: 'rgba(212,168,87,0.8)' },
  separator: {
    height: HIJRI_SEPARATOR_HEIGHT - 4,
    marginVertical: 2,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
    paddingHorizontal: 8,
  },
  sepLine: { flex: 1, height: 1, backgroundColor: 'rgba(212,168,87,0.45)' },
  sepText: { fontFamily: fonts.extrabold, fontSize: 11, letterSpacing: 0.3, color: colors.gold },
  dayRow: { height: MONTH_ROW_HEIGHT, borderRadius: 12 },
  today: {
    backgroundColor: colors.nowRow,
    borderWidth: 1,
    borderColor: colors.nowRowBorder,
  },
  dayCol: { width: DAY_COL },
  dateline: { flexDirection: 'row', alignItems: 'baseline', gap: 4 },
  dateNum: { fontFamily: fonts.extrabold, fontSize: 14, color: colors.text, ...tabularNums },
  weekday: { fontFamily: fonts.semibold, fontSize: 10.5, color: colors.muted },
  hijriDay: { fontFamily: fonts.medium, fontSize: 10, color: 'rgba(212,168,87,0.75)', ...tabularNums },
  gold: { color: colors.gold },
  cell: { flex: 1, textAlign: 'center' },
  time: { fontFamily: fonts.semibold, fontSize: 13, color: colors.text, ...tabularNums },
  timeToday: { fontFamily: fonts.extrabold, color: colors.gold },
});

```

month.tsx src/app/(tabs)

- Изчислява месеца веднъж при смяна (`useMemo`), а часът проверява веднъж в минута – повече не трябва.
- Разделителят се слага преди всеки ден, който е 1-ви по Хиджра (`Fragment` обгражда разделителя и реда).
- Таблицата свършва над долната навигация, за да не се застъпват редовете с нея.

src/app/(tabs)/month.tsx

125 реда

```

import { Fragment, useEffect, useMemo, useRef, useState } from 'react';
import { ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import {
  HIJRI_SEPARATOR_HEIGHT,
  HijriMonthSeparator,
  MONTH_ROW_HEIGHT,
  MonthColumns,
  MonthRow,
} from '@components/MonthTable';
import { MonthHeader } from '@components/MonthHeader';
import { useTabBarHeight } from '@components/TabBar';
import { toHijri } from '@domain/hijri';
import { computeMonth } from '@domain/times';
import { useNow } from '@hooks/useNow';
import { TIMES_OPTIONS } from '@hooks/usePrayerSchedule';

```

```

import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

export default function MonthScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const now = useNow(60_000); // тук е достатъчно веднъж в минута

  const [ym, setYm] = useState(() => ({ y: now.getFullYear(), m: now.getMonth() }));
  const shiftMonth = (delta: number) => {
    setYm(({ y, m }) => {
      const d = new Date(y, m + delta, 1);
      return { y: d.getFullYear(), m: d.getMonth() };
    });
  };

  const days = useMemo(() => computeMonth(ym.y, ym.m, TIMES_OPTIONS), [ym.y, ym.m]);
  const hijri = useMemo(() => days.map((d) => toHijri(d.date)), [days]);

  const isCurrentMonth = ym.y === now.getFullYear() && ym.m === now.getMonth();
  const todayIndex = isCurrentMonth ? now.getDate() - 1 : -1;

  // „Ребиул-ахир - Джемазиел-евел 1448“ (или две години, ако месецът ги пресича)
  const first = hijri[0];
  const last = hijri[hijri.length - 1];
  const hijriRange =
    first.month === last.month
      ? `${t.hijriMonths[first.month - 1]} ${first.year}`
      : first.year === last.year
        ? `${t.hijriMonths[first.month - 1]} - ${t.hijriMonths[last.month - 1]} ${last.year}`
        : `${t.hijriMonths[first.month - 1]} ${first.year} - ${t.hijriMonths[last.month - 1]} ${last.year}`;

  // При отваряне на текущия месец превърта така, че днешният ден да се вижда
  // (с 3 реда над него). Разделителите за нов месец по Хиджра също заемат място.
  const scrollRef = useRef<ScrollView>(null);
  useEffect(() => {
    const target = todayIndex - 3;
    let y = 0;
    for (let i = 0; i < target; i++) {
      y += MONTH_ROW_HEIGHT + (hijri[i].day === 1 ? HIJRI_SEPARATOR_HEIGHT : 0);
    }
    scrollRef.current?.scrollTo({ y, animated: false });
  }, [ym.y, ym.m, todayIndex, hijri]);

  const location = TIMES_OPTIONS.location;

  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      <GeometricPattern />
      <MonthHeader
        title={` ${t.date.monthsFull[ym.m]} ${ym.y} `}
        hijriRange={hijriRange}
        city={location.names[lang]}
        showToday={!isCurrentMonth}
        onPrev={() => shiftMonth(-1)}
        onNext={() => shiftMonth(1)}
        onToday={() => setYm({ y: now.getFullYear(), m: now.getMonth() })}
      />
      {/* Картата свършва над долната навигация, за да не се застъпват редовете с нея. */}
    </View>
  );
}

```

```

<View style={styles.card, { marginBottom: tabBarHeight }}>
  <MonthColumns />
  <ScrollView
    ref={scrollRef}
    showsVerticalScrollIndicator={false}
    style={styles.list}
  >
    {days.map((day, i) => (
      <Fragment key={i}>
        {hijri[i].day === 1 && (
          <HijriMonthSeparator label={` ${t.hijriMonths[hijri[i].month - 1]} ${hijri[i].year}` } />
        )}
        <MonthRow day={day} hijriDay={hijri[i].day} isToday={i === todayIndex} />
      </Fragment>
    ))}
    <Text style={styles.source}>
      {location.source === 'mufti' ? t.month.sourceMufti : t.month.sourceCalc}
    </Text>
  </ScrollView>
</View>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  card: {
    flex: 1,
    marginHorizontal: 10,
    paddingHorizontal: 2,
    paddingTop: 4,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
    overflow: 'hidden',
  },
  list: { paddingBottom: 14 },
  source: {
    fontFamily: fonts.regular,
    fontSize: 11,
    color: colors.muted,
    textAlign: 'center',
    marginTop: 14,
    paddingHorizontal: 16,
  },
});

```

Стъпка 6

Обновяване на календара

Ако Мюфтийството някога промени календара, таблицата се обновява с един скрипт.

update-mufti.mjs tools

- Изпраща същата заявка като формата на сайта за всеки град и месец (48 × 12), маха лятното време и проверява, че всеки град е София + постоянна разлика.
- Ако някой град не следва правилото, спира с грешка, вместо да запише грешни данни.

- **Внимание:** скриптът не е пускан срещу сайта от моята среда (там достъпът до grandmufti.bg е спрян). Пусни го веднъж при теб и ми кажи, ако излезе грешка.

tools/update-mufti.mjs

102 реда

```
// Обновява src/data/mufti.json от сайта на Главно муфтийство.
// Пуска се ръчно, когато Муфтийството промени календара си: node tools/update-mufti.mjs
//
// Как работи:
// 1. За всеки град и всеки месец изпраща същата заявка като формата на сайта (POST month, town).
// 2. Сайтът показва часовете с лятното време вътре – превръщаме всичко в зимно време (UTC+2).
// Датите на смяна откриваме от самата таблица (скок от ~60 мин. при Зухр).
// 3. Проверява, че всеки град = София + постоянна разлика. Ако не е – спира с грешка.
// 4. Записва таблицата на София и разликите; имената и координатите остават от стария файл.
import fs from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

const URL_ = 'https://www.grandmufti.bg/bg/home/vremena-za-namaz.html';
const here = path.dirname(fileURLToPath(import.meta.url));
const OUT = path.join(here, '../src/data/mufti.json');
const DAYS_IN_MONTH = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]; // 29.02 не се пази

const old = JSON.parse(fs.readFileSync(OUT, 'utf8'));
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function fetchMonth(town, month) {
  for (let attempt = 1; attempt <= 4; attempt++) {
    try {
      const res = await fetch(URL_, {
        method: 'POST',
        headers: { 'content-type': 'application/x-www-form-urlencoded' },
        body: `month=${month}&town=${encodeURIComponent(town)}`,
      });
      const html = await res.text();
      const start = html.indexOf('<table');
      if (start < 0) throw new Error('няма таблица');
      const table = html.slice(start, html.indexOf('</table>', start));
      const rows = [...table.matchAll(/<tr[\s\S]*?</tr>/g)]
        .map((m) => [...m[0].matchAll(/<td[\s\S]*?</td>/g)].map((c) => c[0].replace(/<^>+>/g, ''))
        .trim())
        .filter((cells) => cells.length === 7 && Number(cells[0]) >= 1);
      return rows.map((cells) =>
        cells.slice(1).map((t) => {
          const [h, m] = t.split(':').map(Number);
          if (Number.isNaN(h) || Number.isNaN(m)) throw new Error(`лош час „${t}“`);
          return h * 60 + m;
        }),
      );
    } catch (err) {
      console.error(` ${town} ${month}: ${err.message} (опит ${attempt})`);
      await sleep(2000 * attempt);
    }
  }
  throw new Error(`Неуспешно сваляне: ${town}, месец ${month}`);
}

async function fetchYear(town) {
  const rows = [];
  for (let m = 1; m <= 12; m++) {
    const month = await fetchMonth(town, m);
```

```

    if (month.length < DAYS_IN_MONTH[m - 1]) throw new Error(`${town} ${m}: само ${month.length} дни`);
    rows.push(...month.slice(0, DAYS_IN_MONTH[m - 1]));
    await sleep(200);
  }
  return rows;
}

/** Индексите [начало, край] на лятното време в таблицата - там, където Зухр скача с ~60 мин. */
function dstRange(rows) {
  let start = -1;
  let end = -1;
  for (let i = 1; i < rows.length; i++) {
    const jump = rows[i][2] - rows[i - 1][2];
    if (jump > 40) start = i;
    if (jump < -40) end = i;
  }
  if (start < 0 || end < 0) throw new Error('Не открих смяната на часовото време в таблицата');
  return [start, end];
}

const tables = {};
for (const t of old.towns) {
  process.stdout.write(`${t.bg} `);
  tables[t.id] = await fetchYear(t.id);
}
console.log();

const [ds, de] = dstRange(tables.sofia);
for (const id of Object.keys(tables)) {
  tables[id] = tables[id].map((r, i) => (i >= ds && i < de ? r.map((v) => v - 60) : r));
}
const sofia = tables.sofia;
sofia.forEach((r, i) => {
  for (let k = 1; k < 6; k++) if (r[k] <= r[k - 1]) throw new Error(`София, ден ${i + 1}: часовете не са подредени`);
});

const towns = old.towns.map((t) => {
  const diffs = new Set(tables[t.id].flatMap((r, i) => r.map((v, k) => v - sofia[i][k])));
  if (diffs.size !== 1) throw new Error(`${t.bg} не е постоянна разлика спрямо София: ${[...diffs].join(',')} `);
  return { ...t, shift: [...diffs][0] };
});

const changed = JSON.stringify(old.sofia) !== JSON.stringify(sofia) ||
  JSON.stringify(old.towns) !== JSON.stringify(towns);
const out = { ...old, retrieved: new Date().toISOString().slice(0, 10), sofia, towns };
fs.writeFileSync(OUT, JSON.stringify(out));
console.log(changed ? 'Календарът е ПРОМЕНЕН - пусни тестовите (npm test) и провери разликите.' : 'Без промяна в календара.');
```

~5 мин.; после тестовите трябва пак да минат

```

node tools/update-mufti.mjs
npm test
```

Права върху данните: календарът е на Главно мюфтийство. Приложението го посочва като източник в екрана „Месец“. Преди публикуване в магазините е разумно да им пишеш и да поискаш разрешение.

Проверка и край на етапа

Етап 2 е готов, когато:

- `npm test` – 26 от 26 теста минават.
- На „Днес“ часовете за София съвпадат с grandmufti.bg (29.09: 05:45, 07:14, 13:22, 16:41, 19:21, 20:43).
- „Месец“ показва целия месец, днешният ден е в злато, стрелките сменят месеца, а в края на октомври часовете са с час по-рано (зимно време).
- На английски екранът е „Month“ с „Source: Grand Mufti of Bulgaria“.

Терминал

```
git add .  
git commit -m "Етап 2: календар на Мюфтийството, тестове, екран Месец"  
git tag stage-2
```

Ръчните корекции ±минути, корекцията на датата по Хиджра и изборът на град се настройват от екрана „Настройки“ в етап 3. Логиката за тях вече е готова.